AirVoice — Apply Guide (code + how to replace)

Full code for every open item, with exact FIND / REPLACE / ADD / RUN steps · 2026-08-08

How to use this guide

- Do all of this on a **branch**, never directly on main: `git checkout -b fix/audit-apply`.
- Each step is labelled: **FIND** the shown code in the file and **REPLACE** it with the block below it; **ADD FILE** = create a new file with that content; **RUN** = run the SQL/shell command.
- Line numbers are approximate (latest main) — search for the shown code, don't rely on the number.
- After each group, verify: `cd api && npx tsc -p tsconfig.json --noEmit` and `cd web-admin && npx tsc --noEmit && npm run build`.
- Then commit and open a PR into main.

P0 — before next deploy

1. Authenticate the donations endpoints

CRITICAL

File: `api/src/routes/donations.ts`

REPLACE Replace the ENTIRE contents of the file with this:

```
import { FastifyInstance, FastifyRequest } from 'fastify';
import { getSupabase } from '../config/supabase';
import { authenticate, requireFinance } from '../middleware/auth';
interface DonationBody {
  date: string; completed_paid_phone_count: number; percentage?: number;
  chq_no?: string; chq_amount?: number; delivery?: string; sending_date?: string;
}
export default async function (app: FastifyInstance) {
  app.get('/', { preHandler: [authenticate] }, async (_req, reply) => {
    const supabase = getSupabase();
    const { data, error } = await supabase.from('donations').select('*')
      .order('date', { ascending: false });
    if (error) return reply.status(500).send({ error: 'DatabaseError', message: error.message });
    return reply.send(data);
  });
  app.post<{ Body: DonationBody }>('/', { preHandler: [authenticate, requireFinance] },
    async (req: FastifyRequest<{ Body: DonationBody }>, reply) => {
    const supabase = getSupabase(); const body = req.body;
    const { data, error } = await supabase.from('donations').insert([{
      date: body.date, completed_paid_phone_count: body.completed_paid_phone_count,
      percentage: body.percentage ?? 3.0, chq_no: body.chq_no, chq_amount: body.chq_amount,
      delivery: body.delivery, sending_date: body.sending_date }]).select().single();
    if (error) return reply.status(400).send({ error: 'DatabaseError', message: error.message });
    if (body.chq_amount && body.chq_amount > 0) {
      try {
        let catRes = await supabase.from('expense_categories').select('id').eq('name','Donations').single();
        let catId = catRes.data?.id;
        if (!catId) { const n = await supabase.from('expense_categories')
          .insert({ name:'Donations', type:'expense' }).select('id').single(); catId = n.data?.id; }
        if (catId) await supabase.from('expenses').insert({ category_id: catId, amount: body.chq_amount,
          description: `Donation (CHQ: ${body.chq_no || 'N/A'}, Date: ${body.date})`,
          expense_date: body.date, status:'approved', payment_method:'Cheque', submitted_by: req.user?.id });
      } catch (err) { console.error('Failed to create expense for donation:', err); }
    }
    return reply.status(201).send(data);
  });
  app.put<{ Params: { id: string }, Body: DonationBody }>('/:id',
    { preHandler: [authenticate, requireFinance] }, async (request, reply) => {
    const supabase = getSupabase(); const { id } = request.params; const body = request.body;
    const { data, error } = await supabase.from('donations').update({
      date: body.date, completed_paid_phone_count: body.completed_paid_phone_count,
      percentage: body.percentage, chq_no: body.chq_no, chq_amount: body.chq_amount,
      delivery: body.delivery, sending_date: body.sending_date,
      updated_at: new Date().toISOString() }).eq('id', id).select().single();
    if (error) return reply.status(400).send({ error: 'DatabaseError', message: error.message });
    return reply.send(data);
  });
  app.delete<{ Params: { id: string } }>('/:id',
    { preHandler: [authenticate, requireFinance] }, async (request, reply) => {
    const supabase = getSupabase(); const { id } = request.params;
    const { error } = await supabase.from('donations').delete().eq('id', id);
    if (error) return reply.status(400).send({ error: 'DatabaseError', message: error.message });
    return reply.send({ success: true });
  });
}
```

2. Require JWT_SECRET (remove the forgeable default)

HIGH

File: `api/src/middleware/jwtAuth.ts`

FIND Line 5:

```
const JWT_SECRET = process.env.JWT_SECRET || 'airvoice-email-auth-secret-change-in-production';
```

REPLACE With:

```
const JWT_SECRET = process.env.JWT_SECRET;
if (!JWT_SECRET && process.env.NODE_ENV === 'production') {
  throw new Error('FATAL: JWT_SECRET must be set in production');
}
const SECRET = JWT_SECRET || 'dev-only-insecure-secret';
```

REPLACE Then change the two usages — jwt.sign(payload, JWT_SECRET, ...) and jwt.verify(token, JWT_SECRET, ...) — to use SECRET instead of JWT_SECRET.

RUN Add to api/.env.example:

```
JWT_SECRET=change_me_to_a_long_random_string_min_16_chars
JWT_EXPIRES_IN=1h
# generate:  node -e "console.log(require('crypto').randomBytes(48).toString('hex'))"
```

---

3. Block unverified Firebase tokens in production

HIGH

File: `api/src/config/firebase.ts (verifyFirebaseToken)`

FIND The dev-mode branch inside verifyFirebaseToken (starts with):

```
if (!auth) {
  // Dev mode — parse the token claims without verification
  console.warn('[Firebase] Skipping token verification in dev mode');
```

REPLACE Insert the production guard as the FIRST line inside if (!auth) { :

```
if (!auth) {
  if (process.env.NODE_ENV === 'production') {
    throw new Error('Firebase not configured; refusing to verify token in production');
  }
  // Dev mode — parse the token claims without verification
  console.warn('[Firebase] Skipping token verification in dev mode');
  // ...rest of the existing dev block unchanged...
}
```

---

4. Patch vulnerable dependencies

HIGH

File: `api/ and web-admin/ (terminal)`

RUN Run per package, then re-audit:

```
cd api            && npm audit fix
cd ../web-admin   && npm audit fix
cd ../airvoice-help-center && npm audit fix
# xlsx has NO npm patch — replace with the official SheetJS build (in api AND web-admin):
npm remove xlsx
npm i https://cdn.sheetjs.com/xlsx-0.20.3/xlsx-0.20.3.tgz
# major upgrades on a branch, then rebuild + test:
cd api          && npm i fastify@^5
cd ../web-admin && npm i vite@latest && npm run build
npm audit --omit=dev   # repeat until no High/Critical
```

P1 — money-correctness (this sprint)

5. Stop the old/new NIC duplicate-phone fraud

HIGH

File: `NEW api/src/utils/nic.ts + services/customers.ts + SQL`

ADD FILE — Create api/src/utils/nic.ts:

```
export function normalizeNic(raw?: string | null): string | null {
  if (!raw) return null;
  const s = raw.trim().toUpperCase().replace(/\s+/g, '');
  if (/^[0-9]{9}[VX]$/.test(s)) return s.slice(0, 9);   // old: 761234567V -> 761234567
  if (/^[0-9]{12}$/.test(s))    return s.slice(2, 11);  // new: 197612345670 -> 761234567
  const digits = s.replace(/[^0-9]/g, '');
  return digits.length >= 9 ? digits.slice(0, 9) : (digits || null);
}
```

RUN — Run in Supabase (add + backfill the canonical column):

```
ALTER TABLE public.customers ADD COLUMN IF NOT EXISTS nic_canonical text;
CREATE INDEX IF NOT EXISTS idx_customers_nic_canonical ON public.customers(nic_canonical);
UPDATE public.customers SET nic_canonical = CASE
  WHEN new_nic_number ~ '^[0-9]{12}$'      THEN substr(new_nic_number,3,9)
  WHEN nic_number     ~ '^[0-9]{9}[VvXx]$' THEN substr(nic_number,1,9)
  WHEN nic_number     ~ '^[0-9]{12}$'      THEN substr(nic_number,3,9)
  WHEN new_nic_number ~ '^[0-9]{9}[VvXx]$' THEN substr(new_nic_number,1,9) END
WHERE nic_number IS NOT NULL OR new_nic_number IS NOT NULL;
```

FIND — In services/customers.ts — the NIC check inside checkCustomerDuplicates loop (the item with field 'nic_number').

REPLACE — Add a canonical NIC match at the top of checkCustomerDuplicates (keep the other exact checks):

```
import { normalizeNic } from '../utils/nic';
// ...inside checkCustomerDuplicates, before the existing loop:
const canon = normalizeNic(input.nic_number) ?? normalizeNic((input as any).new_nic_number);
if (canon) {
  const { data } = await supabase.from('customers')
    .select('id, full_name').eq('nic_canonical', canon).is('deleted_at', null).limit(5);
  (data ?? []).forEach(d => duplicates.push({ field:'nic_canonical', value:canon,
    existing_customer_id:d.id, existing_customer_name:d.full_name }));
}
// change the checks array to drop 'nic_number' (now covered by canonical) and keep
// service_number / military_id_number / phone_number.
```

REPLACE — In routes/customers.ts POST '/' — set nic_canonical on insert:

```
import { normalizeNic } from '../utils/nic';
const nic_canonical = normalizeNic(body.data.nic_number) ?? normalizeNic(body.data.new_nic_number);
// .insert({ ...body.data, nic_canonical, created_by: request.user!.id })
```

---

6. Add the 3-month rental grace

HIGH

File: `api/src/routes/applications.ts (~line 494)`

FIND — The installment loop:

```
const saleDate = new Date(app.sale_date ?? new Date());
for (let m = 1; m <= app.term_months; m++) {
  const due = new Date(saleDate);
  due.setMonth(due.getMonth() + m);
```

REPLACE — With (first rental now falls on sale + 3 months):

```
const FIRST_RENTAL_OFFSET_MONTHS = 3;   // camp deduction starts 3 months after sale
const saleDate = new Date(app.sale_date ?? new Date());
for (let m = 1; m <= app.term_months; m++) {
```

```
  const due = new Date(saleDate);
  due.setMonth(due.getMonth() + (FIRST_RENTAL_OFFSET_MONTHS - 1) + m);   // m=1 -> sale + 3
```

7. Pay only earned commission into salary, once

HIGH

File: `api/src/routes/payroll.ts + SQL`

RUN Run in Supabase:

```
ALTER TABLE public.commissions
  ADD COLUMN IF NOT EXISTS payroll_run_id uuid REFERENCES public.payroll_runs(id) ON DELETE SET NULL;
CREATE INDEX IF NOT EXISTS idx_commissions_payroll_run ON public.commissions(payroll_run_id);
```

FIND In POST /payroll/runs — the commission fetch:

```
const { data: commissions, error: commissionErr } = await sb.from('commissions')
  .select('sales_officer_id, amount')
  .gte('created_at', monthStart)
  .lte('created_at', `${monthEnd}T23:59:59Z`);
```

REPLACE With (only payable, earned by month end, and reserve them):

```
const { data: commissions, error: commissionErr } = await sb.from('commissions')
  .select('id, sales_officer_id, amount')
  .eq('status', 'payable')
  .is('payroll_run_id', null)
  .lte('became_payable_at', `${monthEnd}T23:59:59Z`);
// after building salesMap, collect includedIds = commissions.map(c => c.id) and:
if (commissions?.length)
  await sb.from('commissions').update({ payroll_run_id: run.id }).in('id', commissions.map(c => c.id));
```

REPLACE In POST /payroll/runs/:id/mark-paid — after it sets payroll_runs.status='paid', add:

```
await sb.from('commissions').update({
  status: 'paid', paid_at: new Date().toISOString(), paid_by: req.user!.id,
}).eq('payroll_run_id', id).eq('status', 'payable');
```

8. Apply the camp filter + camp-wise summary

HIGH

File: `api/src/routes/recovery.ts`

FIND In GET /overdue, the active customers query (pagination fetchAll already added):

```
// the customers fetch currently ignores q.camp_id
```

REPLACE Apply camp_id to that fetch (wherever customers are loaded for /overdue):

```
// when building the customers query, add:
if (q.camp_id) query = query.eq('camp_id', q.camp_id);
// (if it uses fetchAll, pass the apply callback: (qq)=> q.camp_id ? qq.eq('camp_id', q.camp_id) : qq)
```

ADD FILE Add a camp-wise summary endpoint (same file):

```
app.get('/by-camp', { preHandler:[authenticate, requireRole(
  'recovery_officer','finance_officer','admin','super_admin','system_operator')] },
async (_req, reply) => {
  const rows = /* reuse the /overdue result array (arrears>0) */ [];
  const byCamp: Record<string, any> = {};
  for (const r of rows) {
    const k = r.camp?.name ?? 'Unassigned';
    byCamp[k] ||= { camp:k, branch:r.camp?.branch ?? null, customers:0, total_arrears:0, total_missed:0 };
    byCamp[k].customers++; byCamp[k].total_arrears += r.arrears_amount; byCamp[k].total_missed += r.missed_months;
  }
  return reply.send({ data: Object.values(byCamp).sort((a:any,b:any)=> b.total_arrears - a.total_arrears) });
});
```

P2 — hardening

9. CSP, error masking, prod source maps

MEDIUM

File: `api/src/app.ts + web-admin/vite.config.ts`

FIND web-admin/vite.config.ts (~line 23):

```
sourcemap: true,
```

REPLACE With:

```
sourcemap: false,
```

FIND api/src/app.ts — the error handler send():

```
reply.status(error.statusCode ?? 500).send({
  error: error.name,
  message: error.message,
});
```

REPLACE With (mask 5xx in production):

```
const status = error.statusCode ?? 500;
reply.status(status).send({
  error: status >= 500 ? 'Internal Server Error' : error.name,
  message: status >= 500 && process.env.NODE_ENV === 'production'
    ? 'An unexpected error occurred' : error.message,
});
```

For CSP, prefer adding an nginx header (advisory — you asked not to change nginx): `add_header Content-Security-Policy "default-src 'self'; ..." always;`.

---

10. Rate-limit public translate & chat

MEDIUM

File: `api/src/routes/translation.ts, chat.ts`

REPLACE Add a per-route limit to each public route (the app registers @fastify/rate-limit globally):

```
// translation.ts — for each /translate* route:
fastify.post('/translate',
  { config: { rateLimit: { max: 20, timeWindow: '1 minute' } } }, handler);
// chat.ts — the public chat POST:
app.post('/',
  { config: { rateLimit: { max: 15, timeWindow: '1 minute' } } }, handler);
// chat.ts: do NOT inject live DB counts/camp names for anonymous users (fetchSystemContext).
```

---

11. Run containers as non-root (advisory — Docker)

MEDIUM

File: `api/Dockerfile`

REPLACE In the production stage, before CMD:

```
RUN chown -R node:node /app
USER node
# for nginx images use an unprivileged base, e.g. FROM nginxinc/nginx-unprivileged:alpine (expose 8080)
```

---

12. Legacy import: SALES MEMBER -> salesman + 125/125 commission

MEDIUM

File: `api/src/routes/legacy-import.ts + SQL`

This one is large (alias table, parser, resolver, split commission). The complete, ready-to-paste code is in the two delivered PDFs — apply them as-is:

- `AirVoice-Legacy-SalesMember-Commission-Fix.pdf` — alias table, capture SALES MEMBER, Excel-serial sale date, resolve+assign+create commission.
- `AirVoice-SalesMember-Commission-Split-Addendum.pdf` — the confirmed 125+125 split (worker + field salesman by designation).

Final verification & commit

```
cd api            && npx tsc -p tsconfig.json --noEmit
cd ../web-admin   && npx tsc --noEmit && npm run build
cd ../airvoice-help-center && npm run build
git add -A
git commit -m "security & correctness fixes from QA re-audit (P0/P1/P2)"
git push -u origin fix/audit-apply
# open a Pull Request into main; merge after review + green checks
```

Advisory only — no repository files were changed or pushed. Apply on a branch, run the verification above, and merge to main via a reviewed PR.